Ship Real AI

Ship Real AI

The AI Engineer Bootcamp for LLMs, RAG, MCP,
AI Agents and Evals, from First Principles
to Production and Interviews

Sridhar Mukkandi

Draft

Ship Real AI: The AI Engineer Bootcamp for LLMs, RAG, MCP, AI Agents and Evals, from First Principles to Production and Interviews

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

About the examples in this book. Tarkika, its customers and its staff are imagined. Every number about Tarkika, and every price marked as an example, is assumed, to explain the ideas clearly; none is a measurement of a real company or product. Facts about real companies, products and incidents are cited.

Products and services change quickly. Model names, versions and prices were checked on the date given in the version register at the back of the book. All trademarks belong to their respective owners.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or financial advice.

Text set in IBM Plex Serif, Work Sans and IBM Plex Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vi

How to use this book viii

I The job and the foundations xi

1 Python for AI engineers 1

Appendices 14

Answers 14

References 16

About the author 17

Preface

Many people want to become AI engineers, and most of them start the same way. They watch a video, copy a chatbot tutorial, connect it to a vector database, and get a demo working over a weekend. Then they try to put it in front of real users, and it falls apart. The bot invents a refund policy. It costs ten times more than expected. Nobody can say whether last week's change made it better or worse. A customer pastes a strange message, and the bot does something nobody planned.

This book is about the work that starts after the demo. It is a complete path, from the basic probability you need, through how large language models work, to designing AI agents that are safe to run in a real business. It is written for engineers, not for researchers. Every idea is here because you will use it at work or be asked about it in an interview.

Who this book is for

- **You are starting out.** You know some Python, and you want to become an AI engineer. Read the book in order. Part I gives you every foundation the rest of the book needs, and nothing more.
- **You are a software engineer.** You build backends, apps or data pipelines, and now your team wants AI features. Skim Part I and spend most of your time in Parts III to VIII.
- **You work in data science or machine learning.** You know models, but not agents, retrieval or the new protocols. Start at Part II or Part III.

What makes this book different

Most AI books teach tools. Tools change every few months, so those books go out of date quickly. This book teaches the judgement behind the tools, and keeps the details that change often in a dated register at the back.

An AI system is a chain of small decisions. Which team should get this email? Is this the right document? Is it safe to issue this refund, or should a person check first? Each decision needs three things: a number you can trust, a line drawn by what mistakes cost, and a safe way out when the system is unsure. You will meet these three things in Part I, and they come back in every part after it, from model choice to agent design.

You will also build one system from start to finish. **Tarkika** is an imagined online marketplace that sells phones, electronics, fashion and home products across India. Its customers write in English, Hindi and Hinglish. Through the book you will build its customer-support agent: it answers questions from the store's policy documents, looks up orders, and issues refunds behind a human check. By the last chapter, you will have a complete project you can explain in an interview, with every number in it justified.

How the book is organised

1. **The job and the foundations.** What an AI engineer does, and the probability, machine learning, deep learning and language basics underneath it.
2. **How LLMs work.** Transformers, training, reasoning models, long context, serving, and how to choose a model.
3. **Talking to models.** Prompts and context, structured outputs, tool calling, and your first tests.
4. **Knowledge and retrieval.** Embeddings, vector databases, and RAG that works.
5. **Agents.** From fixed workflows to agents, memory, and many agents working together.
6. **Protocols and frameworks.** MCP, A2A, and how to choose a framework.
7. **Trust.** Security, guardrails, private data, and testing agents properly.
8. **Production.** Turning a prototype into a service, running it, cost and speed, fine-tuning, and small models.
9. **Case studies.** Real systems that worked and real systems that failed, told as design reviews.
10. **Career.** Your portfolio, the job market, and interview preparation.

What you need

A laptop, basic Python, and an account with one model provider. Running every example in this book costs very little, and the cost of each one is shown so that nothing surprises you. You do not need a GPU.

Sridhar Mukkandi

How to use this book

Each chapter follows the same shape, so you always know where to find things.

The boxes

This chapter. A dark strip at the start of each chapter, with the three things it covers.

Remember. One line worth keeping, set in italics between two rules. At the end of the chapter, the **Chapter checklist** collects these lines so you can tick them off.

Build it: Tarkika. What we add to the running project in this chapter. If you are building along, do this step before moving on.

Design decision. A real choice an AI engineer has to make, with the reasoning for each option.

In production, this goes wrong. The common ways the chapter's idea fails once real users arrive. These boxes are short, and they are some of the most useful pages in the book.

Defend your numbers. Every AI system is full of numbers someone chose: how many test cases, which threshold, how long a text chunk. In an interview or a design review, you will be asked why. This box lists the numbers from the chapter and how to justify each one.

Practice and Interview corner. Try them before you turn the page. Answers to both are at the back of the book.

Code

Code in this book is short and is written in Python. Each listing runs as printed, once you install the package it imports and supply your own data where the text says so. You do not need to type the code to understand the chapter: the text explains what each part does.

Numbers and prices

Every number about Tarkika is an assumed example, chosen to make the idea easy to see. Prices marked as example prices are made up, to keep the arithmetic simple. Real prices change often, so check your provider’s current price page before you plan a budget.

PART I

The job and the foundations

The first modules give you the ground the rest of the book stands on: the Python habits that keep model calls safe and fast, what an AI engineer builds around a model, and the parts of probability, machine learning and language processing you will actually use at work. None of it is research-level. If an idea is here, a later module or a real job needs it.

01

Python for AI engineers

Ravi is a developer at Tarkika. Customer messages come in by the thousand, and he writes a short Python script to sort them. It reads the messages from a file, sends each one to a language model, and asks for one word back: *refund*, *delivery*, *product* or *other*. He tries it on ten messages. All ten come back right.

Then the Diwali sale starts, and he runs the script on a full day of mail, 4,000 messages. Five things go wrong, one after another.

1. On a colleague's laptop, the script does not even start. An import fails.
2. On his own laptop it starts, but the progress counter says it needs more than three hours.
3. Two hours in, at message 2,180, it stops with an error: `429 Too Many Requests`. Nothing has been saved. Two hours of work are gone.
4. He reruns a small batch and finds that the model does not always reply with one word. Sometimes it says "Refund". Sometimes "refund." with a full stop. Once it says "This customer seems to want their money back." His code was waiting for exactly `refund`.
5. He fixes that. A week later, another small change breaks the script again, and nobody notices for three days, because nothing tests it.

Look at the list again. None of these problems is about how good the model is. Each one is ordinary Python, used in an unusual situation: a slow, unpredictable service that charges per call, sitting in the middle of your program. An AI engineer works in that situation all day. This module takes Ravi's five problems in order and fixes each one.

You need basic Python before you start: variables, functions, loops, lists and dictionaries. If any of those is new, spend a week on a beginner course first. Everything else is explained here.

THIS MODULE

- Keep each project's libraries and secret keys under control, so the code runs the same everywhere.
- Check every reply from a model before your code uses it, and handle failed calls without losing work.
- Make thousands of model calls in minutes instead of hours, and test code whose model never answers the same way twice.

1.1 Why does the script run on one laptop and not on another?

When you run `pip install`, Python puts the library in one folder on your computer, and every project on that computer uses the same folder. Suppose project A was written for version 1 of a library, and project B needs version 2. The folder can hold only one of them. Install version 2 for project B, and project A stops working.

Before reading on: how would you let both projects run on the same laptop?

Give each project its own folder of libraries. Project A's folder keeps version 1, project B's keeps version 2, and neither knows the other exists (Figure 1.1). A private folder like this is called a **virtual environment**. Python can create one for you:

```
python -m venv .venv            # create the environment in a folder called .venv
source .venv/bin/activate       # use it (on Windows: .venv\Scripts\activate)
pip install pydantic pytest     # these now go into .venv only
pip freeze > requirements.txt   # write down the exact versions installed
```

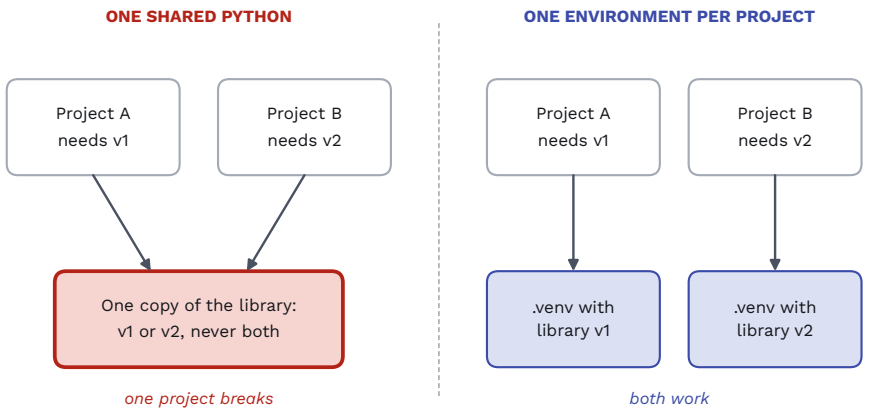


Figure 1.1 Left: every project shares one copy of each library, so two projects that need different versions cannot both work. Right: each project has its own environment with the version it needs.

That fixes one laptop. Now Ravi's colleague makes her own environment and types `pip install pydantic`. She gets the newest version, which may not be the one Ravi tested. The last line above solves this. `pip freeze` writes the exact version of every installed library into `requirements.txt`. She runs `pip install -r requirements.txt` and gets the same versions Ravi has. Fixing a project to exact versions like this is called **pinning**.

Pinning matters more in AI work than in most programming. The libraries for model providers and agent frameworks change quickly, and functions are renamed or removed between versions. A project without pinned versions can break on a new laptop even when nobody has touched its code.

There is one more thing to fix before the code leaves Ravi's laptop. To call a model, the script needs an **API key**: a secret string that proves the call comes from Tarkika's account, and decides who pays for it. Ravi typed the key straight into his script. If that script is pushed to a shared repository, everyone with access can copy the key and spend Tarkika's money. Keep the key outside the code, and let the code read it when it runs:

```
import os
api_key = os.environ["MODEL_API_KEY"]   # set on each machine, never written in the
   code
```

Each person sets `MODEL_API_KEY` on their own machine. Many projects keep such values in a file called `.env`. If you do, add `.env` to `.gitignore`, so the file can never be committed by mistake.

REMEMBER

Give each project its own environment, pin the versions you tested, and keep keys out of the code.

The script now runs on both laptops. It still has to make sense of what the model sends back, and that is where problem 4 comes from.

1.2 Why does the model's reply break the code that reads it?

Ravi asked the model for one word, and got "Refund", "refund." and a full sentence. His code compared the reply with the exact text `refund`, so all three failed.

The two sides behave very differently. Ravi's code is strict: `"Refund" == "refund"` is false. The model is loose: it writes language, and language allows the same meaning in many forms. Somewhere between the two, something has to check the reply before the strict side uses it.

Before reading on: for Ravi's script, what exactly would you check in each reply?

A reasonable list looks like this. The category must be one of the four allowed words. If the customer mentioned an order number, it should be there. And Ravi also wants to know whether the message is urgent, which must be a plain yes or no. That list is a description of the shape you expect. Write it down as code, and you can check every reply against it automatically. A description like this is called a **data model**, and the library most Python AI projects use for it is Pydantic:

```
from typing import Literal
from pydantic import BaseModel, ValidationError

class Ticket(BaseModel):
    category: Literal["refund", "delivery", "product", "other"]
    order_id: str | None = None
    urgent: bool

reply = '{"category": "Refund request", "urgent": true}'
try:
    ticket = Ticket.model_validate_json(reply)
except ValidationError as e:
    print(e.errors()[0]["msg"])
```

Each line of `Ticket` is one rule. `Literal[...]` allows only those four words. `str | None = None` means the order number is text, or missing. `bool` means true or false. The reply here is in JSON, a standard text format for structured data; Module 7 shows how to ask a model for it. The model has filled in "Refund request" as the category. Running the code prints:

```
Input should be 'refund', 'delivery', 'product' or 'other'
```

The bad reply is stopped at the door, with a clear reason, instead of causing a confusing error three functions later.

The annotations such as `str | None` and `bool` are **type hints**: notes that say what kind of value each name holds. In most Python code they are only notes, read by people and

code editors; Python itself does not enforce them. Pydantic does enforce them. And in AI work, type hints have one more reader, which surprises most people coming from ordinary software: the model itself.

```
def check_order_status(order_id: str) -> str:
    """Return the delivery status of one Tarkika order.

    order_id: the order number, for example TK-48213.
    """
```

A function the model may ask your program to run is called a **tool**. When you hand a function like this to an agent framework as a tool, the framework reads its name, its type hints and its docstring. It turns them into a description and sends that to the model. Many frameworks do exactly this (OpenAI 2025). The model decides when to use the tool, and what to pass it, from that description alone. So a vague docstring is not just untidy. It is a wrong instruction to the model. Module 7 returns to this.

REMEMBER

Describe the shape you expect, and check every model reply against it before your code uses it.

Checking catches bad replies. It cannot help when no reply comes at all, which is what happened at message 2,180.

1.3 What should the script do when the model's server says no?

Every call to a model travels over the internet to computers that belong to someone else. Some calls fail. For each kind of failure, ask one question: would waiting fix it?

- **429, too many requests.** You sent more calls than your account allows in a short time. Waiting fixes it.
- **500 or 529, server error or overloaded.** The provider's machines are struggling. Waiting usually fixes it.
- **A timeout.** No reply came in time. Trying again usually works.
- **400, bad request.** Something in your call is wrong: the text is too long, or a field is missing. Waiting will never fix it.
- **401, unauthorised.** The key is wrong or has been revoked. Waiting will never fix it either.

So the first rule is to retry only the first three kinds. Retrying the others repeats the same mistake, wastes time, and can even cost money.

Before reading on: after a 429, how long should the script wait before it tries again?

If it tries again at once, the server is still busy, and the call fails again. Now imagine a thousand scripts, all told "wait one second, then retry". They all come back at the same moment, and the server is flooded again, like a crowd that surges every time a gate opens. Two changes fix this. First, wait longer after each failure: up to 1 second, then 2, then 4, then 8. Second, choose a random wait within that limit, so that different scripts come back at different moments. This is called **exponential backoff with jitter** ("exponential" because the limit doubles each time, "jitter" for the randomness) (Brooker 2015). Stop after a fixed number of tries, and record the failure instead of trying for ever. When the server sends a `retry-after` header saying how long to wait, use that instead.

```
import random, time

RETRYABLE = {429, 500, 503, 529}

def call_with_retry(send, max_attempts=5):
    for attempt in range(max_attempts):
        if attempt > 0:
            time.sleep(random.uniform(0, 2 ** (attempt - 1)))   # full jitter
        status, body = send()
        if status == 200:
            return body
        if status not in RETRYABLE:
            raise RuntimeError(f"Not retryable: {status}")
    raise RuntimeError("Gave up after retries")
```

Here `send` stands for any function that makes one call and returns its status code and body. Real client libraries raise an exception with the status code instead of returning it, but the logic is the same. Figure 1.2 shows the decision as a picture.

Now put numbers on it. Five attempts have four waits between them, of at most 1, 2, 4 and 8 seconds:

1. Worst case before giving up: 1 + 2 + 4 + 8 = 15 seconds of waiting.
2. With random waits, the average is about half of that: around 7 or 8 seconds.

For an overnight batch, 15 seconds is nothing. For a customer waiting in a chat window, it is far too long, and fewer attempts are better. The right number depends on who is waiting.

There is a trap here. The official client libraries of the major model providers already retry failed calls a few times on their own. If the library makes up to 3 attempts and your loop makes 5, one stubborn message can be sent 3 × 5 = 15 times. Check what the library does before you add a loop of your own.

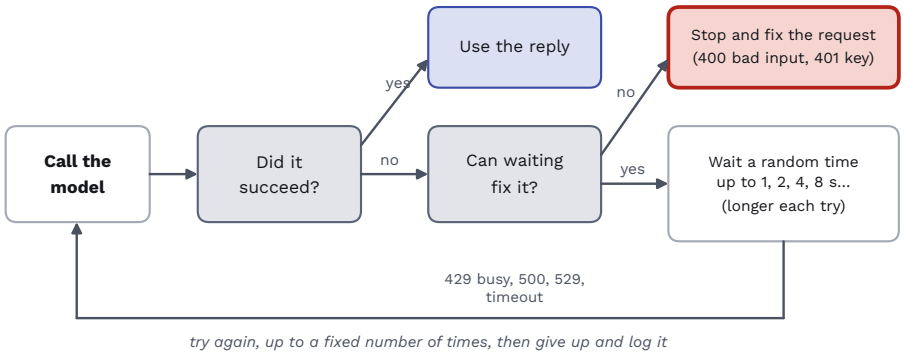


Figure 1.2 What to do with a failed call. Only failures that waiting can fix are retried, with a longer and partly random wait each time.

Retries make crashes rarer. They cannot remove them, and Ravi lost two hours because his script kept every result in memory until the very end. The fix is simple: write each result to a file as soon as it arrives, one line per message. When the script starts, it reads that file and skips the messages already done. A crash at message 2,180 then costs one message, not two hours. Module 16 builds the same idea into long-running agents.

REMEMBER

Retry only what waiting can fix, wait longer and a little randomly each time, and save each result as soon as you have it.

The script no longer loses work. It is still painfully slow, and the reason is not where most people look first.

1.4 Why do 4,000 messages take three hours?

Each model call takes about 3 seconds. One after another, 4,000 calls take 4,000 × 3 = 12,000 seconds, which is 3 hours and 20 minutes.

Before reading on: of those 3 seconds, how much time is Ravi's computer actually working?

Almost none. The script sends a few kilobytes of text and then waits. The real work happens on the provider's machines. For about 3 seconds, Ravi's program does nothing but wait, and it does that 4,000 times in a row.

Think of someone in Tarkika's support team who emails a courier company about one lost parcel. Then they stare at the inbox until the courier replies, and only then write about the next parcel. Nobody works like that. You send twenty emails, and handle the replies as they arrive. Python can work the same way:

```
import asyncio

async def ask_model(message):          # stand-in for a real model call
    await asyncio.sleep(3)             # the time spent waiting for the reply
    return "refund"

async def classify_all(messages, limit=20):
    gate = asyncio.Semaphore(limit)
    async def one(m):
        async with gate:
            return await ask_model(m)
    return await asyncio.gather(*(one(m) for m in messages))

results = asyncio.run(classify_all(["paisa wapas karo"] * 100))
```

An `async def` function can pause in the middle. Each `await` marks a place where it pauses while it waits, and lets other calls run in the meantime. `gather` starts all the calls and collects their results. The **semaphore** is a counter that lets only 20 calls be in progress at any moment; the 21st waits until one finishes. Run it, and the 100 stand-in messages finish in about 15 seconds instead of 300. (`ask_model` here only pretends to call a model, so you can run the example without a key. Module 2 makes the real call.)

Having many tasks in progress at once, with one moving forward while another waits, is called **concurrency**. It is different from **parallelism**, where several processor cores calculate at the same instant. Model calls need concurrency, because their time goes in waiting, not in calculating (Figure 1.3).

Why not start all 4,000 at once, then? Because the provider limits how many calls your account may make per minute, and how much text it may send. Do the sum for 20 calls at a time, each taking 3 seconds:

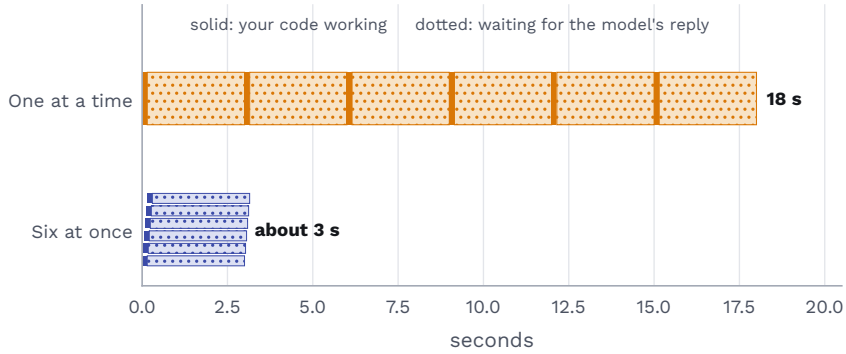


Figure 1.3 Six calls of 3 seconds each. One at a time, the program spends almost all of its 18 seconds waiting. Started together, the six waits overlap and the whole batch takes about 3 seconds.

1. Each of the 20 slots finishes 60 ÷ 3 = 20 calls a minute.
2. Twenty slots finish 20 × 20 = 400 calls a minute.
3. Suppose Tarkika's account allows 100 calls a minute. Then 4,000 messages need at least 4,000 ÷ 100 = 40 minutes, however clever the code.

Push above the limit and you get 429 errors, which your code now retries, which adds even more calls. So the limit should come from the account's allowance, not from how fast you would like to finish. Here 5 slots × 20 calls a minute = 100 a minute, which fits exactly.

REMEMBER

Model calls spend their time waiting, so run several at once, but never more than your account's limit allows.

Three hours have become forty minutes, and the script survives errors. One problem is left: the change that broke it silently for three days.

1.5 How do you test code when the model answers differently every time?

The usual answer to silent breakage is tests. An ordinary unit test gives a function a fixed input and checks for one exact output. Try that here: send the model "paisa wapas karo" and check that it returns `refund`. The test passes today. Tomorrow the model writes "Refund", or the provider updates the model, and the same test fails although nothing in your code changed. A test that fails at random gets ignored, which is worse than having no test.

Before reading on: which parts of Ravi's script behave exactly the same every time?

Everything Ravi wrote himself does: reading the file, checking the shape, retrying, saving results. Only the model's reply varies. So split the testing in two.

1. **Test your own code with a fake model.** A fake is a few lines that return replies you choose, including broken ones, instead of calling the real model. Your code is then tested with ordinary tests, and every test gives the same result every time.
2. **Measure the real model separately.** Run it on a set of real messages and check properties of each reply, such as "is the category one of the four?" The result is a score, not a pass or fail. These checks are called **evals**, and Module 11 is about them.

To make the first kind easy, put the checking step in its own function:

```
class BadReply(Exception):
    pass

def parse_ticket(reply: str) -> Ticket:
    try:
        return Ticket.model_validate_json(reply)
    except ValidationError as e:
        raise BadReply(reply) from e
```

Then test it with `pytest`, the most common testing tool in Python:

```
import pytest
from triage import parse_ticket, BadReply

def test_valid_reply():
    t = parse_ticket('{"category": "delivery", "urgent": false}')
    assert t.category == "delivery"

def test_reply_that_is_not_json():
    with pytest.raises(BadReply):
        parse_ticket("Sure! This looks like a delivery issue.")
```

These tests run in a fraction of a second, with no network, no key and no cost. Fakes also let you test what is hard to produce on demand: a burst of 429 errors, a reply cut off halfway, a server that never answers.

REMEMBER

Test your own code against a fake model, and measure the real model separately with evals.

1.6 Ravi's script, rebuilt

Figure 1.4 puts the five fixes together. Each step answers one of the problems from the start of the module.

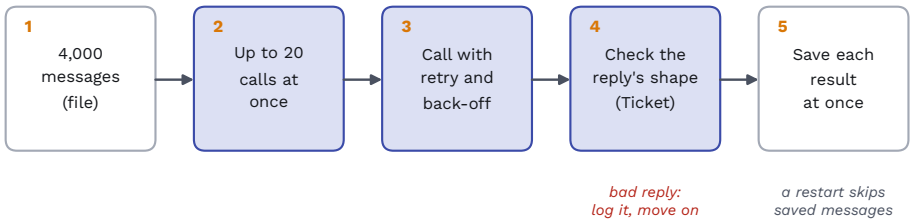


Figure 1.4 The rebuilt triage script. Messages go out up to 20 at a time, failed calls are retried with back-off, every reply is checked against the Ticket model, and each result is saved as soon as it arrives.

1. A pinned environment, so it runs the same on every laptop (problem 1).
2. Up to 20 calls at once, within the account's limit (problem 2).
3. Retries with back-off for failures that waiting can fix (problem 3).
4. Every reply checked against `Ticket`; bad replies logged and skipped (problem 4).
5. Each result saved at once, so a restart loses nothing; and tests for all of Ravi's own code (problems 3 and 5).

None of this needed a smarter model. It needed ordinary engineering around the model, and that is most of what this book is about.

BUILD IT: TARKIKA

Build the first piece of Tarkika's support system: the triage script.

1. Create a project with its own environment. Install `pydantic` and `pytest`, and pin them in `requirements.txt`.
2. Write the `Ticket` model and `parse_ticket`. Add tests for at least four broken replies: not JSON, an unknown category, a missing `urgent` field, and an empty string.
3. Write `ask_model` as a stand-in that waits a random 1 to 4 seconds and sometimes fails with a 429. Wrap it in `call_with_retry`.
4. Classify 200 made-up messages with at most 10 calls at a time. Save each result to `results.jsonl` as it arrives, stop the script halfway, start it again, and check that it continues where it stopped.

In Module 2 you will replace the stand-in with a real model call.

⚠ IN PRODUCTION, THIS GOES WRONG

- **Retries on top of retries.** Your loop and the client library both retry, and one failing message turns into fifteen calls.
- **A key in the logs.** Error messages or debug prints include the full request, key and all, and the logs are shared.
- **Unpinned libraries.** A new release changes a function, and a script nobody touched fails on the next deploy.
- **No cap on concurrency.** Thousands of calls start at once, all get 429 errors, all retry together, and the batch takes longer than running one at a time.
- **Results kept only in memory.** A crash near the end throws away hours of paid calls.

DEFEND YOUR NUMBERS

- **Concurrency.** "We run 5 calls at a time. Each takes about 3 seconds, so that is 100 a minute, which is our account's limit."
- **Retries.** "Five attempts with doubling back-off and jitter, so at most 15 seconds of waiting. That is fine for the overnight batch. In the chat window we allow two attempts, because the customer is waiting."
- **Pinned versions.** "Every library is pinned. We upgrade on purpose, one library at a time, and run the tests after each upgrade."

MODULE CHECKLIST

- ☐ Give each project its own environment, pin the versions you tested, and keep keys out of the code.
- ☐ Describe the shape you expect, and check every model reply against it before your code uses it.
- ☐ Retry only what waiting can fix, wait longer and a little randomly each time, and save each result as soon as you have it.
- ☐ Model calls spend their time waiting, so run several at once, but never more than your account's limit allows.
- ☐ Test your own code against a fake model, and measure the real model separately with evals.

Practice

1. A batch of 6,000 messages, each call taking 4 seconds, with 10 calls at a time. How long does it take? How many calls a minute is that, and would it fit within a limit of 120 a minute?
2. For each of these, would you retry? A 401 error; a timeout; a 400 error saying the text is too long; a 529 error.
3. Write a `Ticket`-style model for a delivery complaint that must include an order number, a city, and how many days late the parcel is (a whole number from 0 to 60).

Interview corner

1. How would you call an LLM API 10,000 times efficiently?
2. Which API errors would you retry, and how?
3. How do you write unit tests for code that calls an LLM?
4. Why do you need to validate the output of an LLM?

Up next

The script now sorts 4,000 messages safely. Next: what the model inside it actually is, what an AI engineer builds around it, and what each of those 4,000 calls costs in rupees.

Answers

Some exercises have one right answer. Others ask about your own work; for those, you will find a sample answer and what a good answer includes. Interview answers are short on purpose: they show the points an interviewer listens for, which you should then back with an example from your own project.

Module 1

Exercises

1. 6,000 ÷ 10 = 600 rounds of 4 seconds each, so 2,400 seconds, or 40 minutes. Each of the 10 slots finishes 60 ÷ 4 = 15 calls a minute, so 150 calls a minute in all. That is above a limit of 120, so you would get 429 errors. Eight slots give 8 × 15 = 120 calls a minute, and the batch then takes 6,000 ÷ 120 = 50 minutes.
2. 401: no, fix the key. Timeout: yes, with back-off. 400 "text too long": no, shorten the text. 529: yes, with back-off.
3. A sample answer:

```python
from pydantic import BaseModel, Field

class DeliveryComplaint(BaseModel):
    order_id: str
    city: str
    days_late: int = Field(ge=0, le=60)
```

   A good answer makes `order_id` and `city` required (no default), and uses `Field(ge=0, le=60)` so that a reply of 75 or −2 is rejected.

Interview questions

1. Run the calls concurrently with async code, and cap how many are in progress with a semaphore. Set the cap from the account's rate limit, not from the hoped-for speed. Retry failures that waiting can fix, with back-off and jitter. Save each result as it arrives, so that a crash does not lose the batch. If the results are not needed at once, a provider's batch API is cheaper still.
2. Retry 429, 5xx errors and timeouts, with exponential back-off, random jitter and a maximum number of attempts, and honour a `retry-after` header when there

is one. Do not retry 400, 401, 403 or 404, because the request itself must change. Check whether the client library already retries, so that retries are not multiplied.

3. Separate the code you wrote from the model. Test your code with a fake model that returns fixed replies, including broken ones and errors, so the tests are fast, free and give the same result every time. Measure the model's behaviour separately with evals on a set of real examples.
4. The model produces language, and the same meaning can come in many forms, or in the wrong form. Code needs exact values. Checking every reply against a data model, such as a Pydantic class, catches bad replies at once with a clear reason, instead of letting them cause errors later.

References

Brooker, Marc. 2015. *Exponential Backoff and Jitter*. AWS Architecture Blog. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/.

OpenAI. 2025. *OpenAI Agents SDK: Function Tools*. Documentation. https://openai.github.io/openai-agents-python/tools/.

About the author

Sridhar Mukkandi is an applied AI engineer who builds AI agents and the decision systems behind them. He writes for engineers who want AI systems they can trust in production.

sridhar.authorhub@gmail.com